RAG: Baseline Retrieval vs. Cross-Encoder Re-Ranking

Suresh Kumar

Code: https://github.com/Suresh-Kumar-Vuyala/RAG_LMA

Setup

Component	Choice
Document	MCC *Laws of Cricket*, 2017 Code, 4th Edition (2026), 90 PDF pages
Chunking	One chunk per numbered section (e.g. *36.1 Out LBW*); sections over 180 words split with 30-word overlap; 392 chunks
Bi-encoder	`sentence-transformers/all-MiniLM-L6-v2`, FAISS `IndexFlatIP` on normalised vectors (cosine similarity), Top 10
Cross-encoder	`cross-encoder/ms-marco-MiniLM-L-6-v2`, re-scores the same Top 10
LLM	Gemini 3.5 Flash, temperature 0, identical prompt for both pipelines (answer only from context, otherwise "Not found")

1 Architecture Comparison

Bi-encoder takes the question alone and converts it into a vector. Similarly, it takes each chunk alone and converts them into vectors. Then it compares how close these vectors are using cosine similarity. Since each chunk is encoded separately, we can calculate their vectors in advance, so searching becomes very fast. The main drawback is that the question and chunk are never processed together. Both are converted into a single vector, so some small details like "not", "except" or who is doing something can get lost.

Cross-encoder takes the question and the chunk together as a single input. Every word in the question can attend to every word in the chunk, and then it gives a relevance score. It is more accurate because it can understand the relationship between the question and the chunk better. But it has to run for every question and chunk pair whenever a query comes, so nothing can be precomputed in advance, which makes it slower.

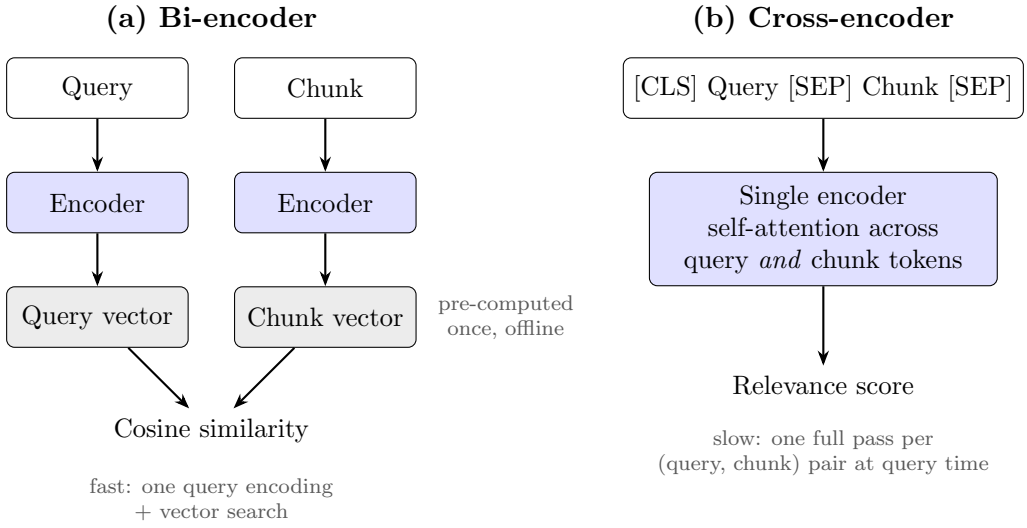


Figure 1: (a) The bi-encoder encodes the query and each chunk separately and compares the two vectors. (b) The cross-encoder reads the query and chunk together as one input and outputs a single relevance score.

2 Qualitative & Rank-Shift Analysis

Table 1 lists every test query. *Rank shift* is the best rank of a gold chunk (the chunk that actually answers the question) before and after re-ranking. The LLM only sees the Top 3.

#	Failure type	Query	Gold	Rank shift	Answer 1	Answer 2
1	Keyword trap	What is the penalty for the fielding side wasting time?	41.9	#8 → #1	Not found	Correct
2	Synonym gap	Can the fielding side use saliva or sweat to shine the ball?	41.3	#2 → #1	Correct	Correct
3	Exception	Is a batter out LBW if the ball pitched outside leg stump?	36.1	#5 → #1	Not found	Correct
4	Negation	Can a fielder who has just come back onto the field bowl straight away?	24.2 (part 2)	#3 → #1	Correct	Correct
5	Buried chunk	Are runs scored if the batters run after the ball hits the striker's body without any attempt to play it?	23.2	#5 → #1	Not found	Correct
6	Buried chunk	Is a run counted if the batter fails to ground their bat beyond the popping crease when turning?	18.3	#4 → #1	Not found	Not found
7	Limitation	Is a bowler allowed to deliberately bowl a high full toss at the batter?	41.7 (parts 1, 3)	#7 → #7	Not found	Not found
8	Limitation	Is a batter allowed to wilfully damage the ball?	41.3	not in Top 10	Not found	Not found
Gold chunk in Top 3					**2 / 8**	**6 / 8**

Table 1: Rank shifts and answer outcomes (Answer 1 = baseline Top 3, Answer 2 = re-ranked Top 3).

Case discussion

Case 1 – Keyword trap. In the first case, the main issue was the keyword trap. The question was about the fielding side wasting time, but the baseline model returned "Runs scored" (37.5), "Penalty runs" (41.17) and "Striker in protected area" (41.15) as the top 3 results, with "Batter wasting time" (41.10) just behind at positions 4 and 5. The correct chunk, Law 41.9, was ranked at positions 8 and 10. This happened because the bi-encoder mainly matched the question with chunks containing similar words and topics like penalty, wasting time and runs. Since "batter wasting time" and "fielding side wasting time" have similar meanings, the model was unable to properly identify the difference between who was wasting time. The cross-encoder handled this better because it processed the question and the chunk together. It identified the matching reference to the fielding side in Law 41.9 and ranked it first. As a result, Answer 1 returned "Not found", whereas Answer 2

provided the correct rule. According to the rule, the fielding side receives a first and final warning, and if the offence is repeated, 5 penalty runs are awarded to the batting side if it is not during an over, or the bowler is suspended if it occurs during an over.

Case 3 – Exception. In the third case, the question was about an exception in the LBW rule. The baseline model returned "Out from a Wide", "Ball bouncing... No ball" and "Out Stumped" as the top three results, while the correct LBW law, Law 36.1, was ranked at position 5. This happened because different dismissal rules use similar wording, so the bi-encoder treated them as similar. It failed to focus on the specific condition mentioned in the question, which was the ball pitching outside leg stump. Since it mainly considered the overall similarity between the question and the chunks, it missed the important detail. Law 36.1.2 states that the ball must pitch in line between the wickets or on the off side. So, if the ball pitches outside leg stump, the batter cannot be given out LBW. Answer 2 was able to retrieve this clause and give the correct explanation, whereas Answer 1 failed to find the relevant rule and returned "Not found".

Cases 5 & 6 – Buried chunks. In cases 5 and 6, the main issue was that the questions described a situation without using the official name of the rule. For example, when the ball hits the batter's body and the batters run, it relates to a leg bye. Similarly, when a batter does not ground the bat while turning, it relates to a short run. The bi-encoder struggled because the correct chunks used different words from the questions. As a result, they were ranked at positions 5 and 4, behind chunks like "Runs scored" and "When out of their ground", which were closer in wording or topic to the questions. The cross-encoder handled this better by reading the question and the chunk together, understanding the situation and moving both correct chunks to the first position. In Case 5, the re-ranking helped retrieve the correct rule. According to Law 23.2.1, the runs count only if the batter tried to play the ball or tried to avoid being hit. However, Case 6 showed a different problem. Although Law 18.3 was moved to the first position, the answer was still "Not found". This is because Law 18.3 only defines a short run, while Law 18.4 explains that a short run is not scored. Since Law 18.4 was not even in the top 10 results, the system could not retrieve the complete information needed to answer the question. This shows that better ranking alone is not always enough. Sometimes, the information needed to answer a question is spread across multiple sections, and retrieving only one of them is not sufficient. In this case, the problem was with chunking and retrieval rather than the cross-encoder itself.

Cases 2 & 4 – Rank improved, answer unchanged. In Cases 2 and 4, the main observation was that improving the ranking did not change the final answer. The correct chunk was already at position 2 in the saliva case and position 3 in the fielder returning case, so it was included in the top 3 results. This means the LLM could access the correct information in both cases, and both answers were already correct. After re-ranking, the correct chunks moved to the first position, but the final answers remained the same. This shows that improving the rank does not always improve the answer. What matters here is whether the correct chunk is included in the top 3 results, rather than its exact position within those results. In Case 2, the baseline model ranked "Watering the pitch" first. Since both saliva and water are liquids, the bi-encoder probably considered their meanings similar. However, the correct saliva-related chunk was still present in the top 3, so the LLM was able to provide the correct answer.

Cases 7 & 8 – Where re-ranking cannot help. In Cases 7 and 8, re-ranking was not able to improve the results because the correct information was either not retrieved or remained too low in the ranking. In Case 8, the correct sentence about wilfully damaging the ball was present in Law 41.3, which deals with the condition of the ball. However, it did not appear in the top 10 results. Instead, both pipelines returned chunks related to "Batter damaging the pitch", likely because they shared words like "batter" and "damaging". Since the re-ranker can only reorder the chunks retrieved in the first stage, it cannot find the correct information if that chunk is missing from the results. This means the performance of the second stage depends heavily on what the first stage retrieves. In Case 7, the question was about a high full toss, but the relevant rule in Law 41.7 describes it as a non-landing delivery above waist height. The definition and the rule about deliberate delivery remained at position 7 or lower in both pipelines. Even the cross-encoder failed to connect the different wording used in the question and the rulebook. These cases show that both

models can struggle when the wording of a question is very different from the wording in the source document. One possible improvement is to retrieve more candidates, such as the top 30 instead of the top 10. We could also use a stronger embedding model or combine keyword search with vector search to improve the chances of retrieving the correct chunks in the first stage.

Type of baseline error. The main thing we observed with the baseline model was that it did not make up any information. Whenever the correct chunk was missing from the retrieved results, it simply returned "Not found". So, the main type of error was omission, where the model failed to provide an answer even though the required information was available in the document. The strict prompt also played an important role here. Gemini already has knowledge about cricket, so without the instruction to answer only from the given context, it could have answered using its existing knowledge. In that case, Answer 1 might have looked correct even when the retrieved chunks were wrong, making it difficult to identify the actual retrieval problem. By restricting the model to the provided context, we can better evaluate how retrieval affects the final answer. In a real-world system, allowing the LLM to use its own knowledge might seem helpful, but it also creates a risk. When the retrieval system misses the correct information, the LLM might still give a confident answer without any supporting source. This makes it harder to identify retrieval failures and verify whether the answer is actually supported by the document.

3 Latency Profiling

Retrieval timings are the mean of 5 runs per query over the 8 test queries (after a warm-up run), on a laptop CPU. LLM time is the mean over the Gemini calls.

Stage	**Pipeline 1 (ms)**	**Pipeline 2 (ms)**
Bi-encoder search (embed query + Top 10)	12.9	12.9
Cross-encoder re-rank (10 pairs)	–	247.6
LLM generation (Gemini)	4,726.6	4,726.6
Retrieval only	**12.9**	**260.5**
End-to-end	**4,739.4**	**4,987.0**

Table 2: Mean latency per query.

The retrieval time increased from 12.9 ms to 260.5 ms after adding the cross-encoder, making the retrieval process around 20 times slower. However, the overall response time increased only from 4.74 seconds to 4.99 seconds, which is about a 5% increase. The reason for this small increase in the overall time is that most of the time is spent on the LLM call, which takes around 4.7 seconds. Since Gemini runs on Google's servers, the network communication also adds to the response time. Compared to this, the additional time taken by the re-ranker has a much smaller effect on the overall latency. In my opinion, the extra latency is worth it because the improvement in retrieval quality is significant. The correct chunk reached the top 3 in 6 out of 8 cases, compared to only 2 out of 8 with the baseline. Also, 3 answers that previously returned "Not found" became correct after re-ranking. Considering these improvements, an additional delay of around 0.25 seconds seems reasonable for a system where answer accuracy is more important than a small increase in response time. One thing to consider is that these measurements were taken on a laptop CPU, so the results may vary between runs. In an earlier run, the re-ranking step took around 200–210 ms. Using a GPU could also reduce the re-ranking time significantly.

4 Scaling Question

The main issue with the cross-encoder is that it needs to run the model separately for every question and chunk pair whenever a new question comes in. Since it needs both the question and the

Approach	**Time per query**
Bi-encoder search over all 392 chunks	10.1 ms
Cross-encoder over Top 10 only	247.6 ms
Cross-encoder over all 392 chunks	6.5 s
Cross-encoder over 10,000 chunks (est.)	$\approx$ 2.8 min
Cross-encoder over 1,000,000 chunks (est.)	$\approx$ 4.6 h

Table 3: Measured and extrapolated cost (16.7 ms per query–chunk pair on this machine; estimates assume linear scaling).

chunk together to calculate the relevance score, the results cannot be precomputed in advance. In comparison, the bi-encoder computes the vectors for all chunks only once. When a question comes in, it only needs to encode the question and compare its vector with the stored chunk vectors. This makes the retrieval process much faster, taking around 10 ms even when comparing against all the chunks. In our experiment, the cross-encoder took around 16.7 ms per question and chunk pair. For 392 chunks, the total time was around 6.5 seconds. Since the computation grows linearly with the number of chunks, processing 10,000 chunks would take approximately 2.8 minutes, while processing 1 million chunks would take around 4.6 hours for a single question. This makes using the cross-encoder on every chunk impractical for large datasets. To overcome this problem, a two-stage retrieval approach can be used. First, the bi-encoder quickly searches through a large number of chunks and selects a small set of relevant candidates. Then, the cross-encoder processes only these candidates and ranks them more accurately. This approach combines the speed of the bi-encoder with the better ranking ability of the cross-encoder, giving improved accuracy without the high computational cost of running the cross-encoder on every chunk.